Task_1:

```python
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import numpy as np
import random

class Environment:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.obstacles = np.zeros((height, width), dtype=bool)
        self.buildings = []
        self.houses = []
        self.delivery_points = []
        self.vehicles = []
        self.start_position = (0, 0)  # Set the initial start position

        # Randomly generate buildings and houses
        self.generate_obstacles()

    def generate_obstacles(self):
        # Generate random positions for buildings
        num_buildings = random.randint(5, 10)
        for _ in range(num_buildings):
            x = random.randint(0, self.width - 1)
            y = random.randint(0, self.height - 1)
            self.add_building(x, y)

        # Generate random positions for houses
        num_houses = random.randint(5, 10)
        for _ in range(num_houses):
            x = random.randint(0, self.width - 1)
            y = random.randint(0, self.height - 1)
            self.add_house(x, y)

    def add_building(self, x, y):
        self.buildings.append((x, y))
        self.obstacles[y, x] = True

    def add_house(self, x, y):
        self.houses.append((x, y))
        self.obstacles[y, x] = True

    def add_delivery_point(self, x, y):
        self.delivery_points.append((x, y))
```

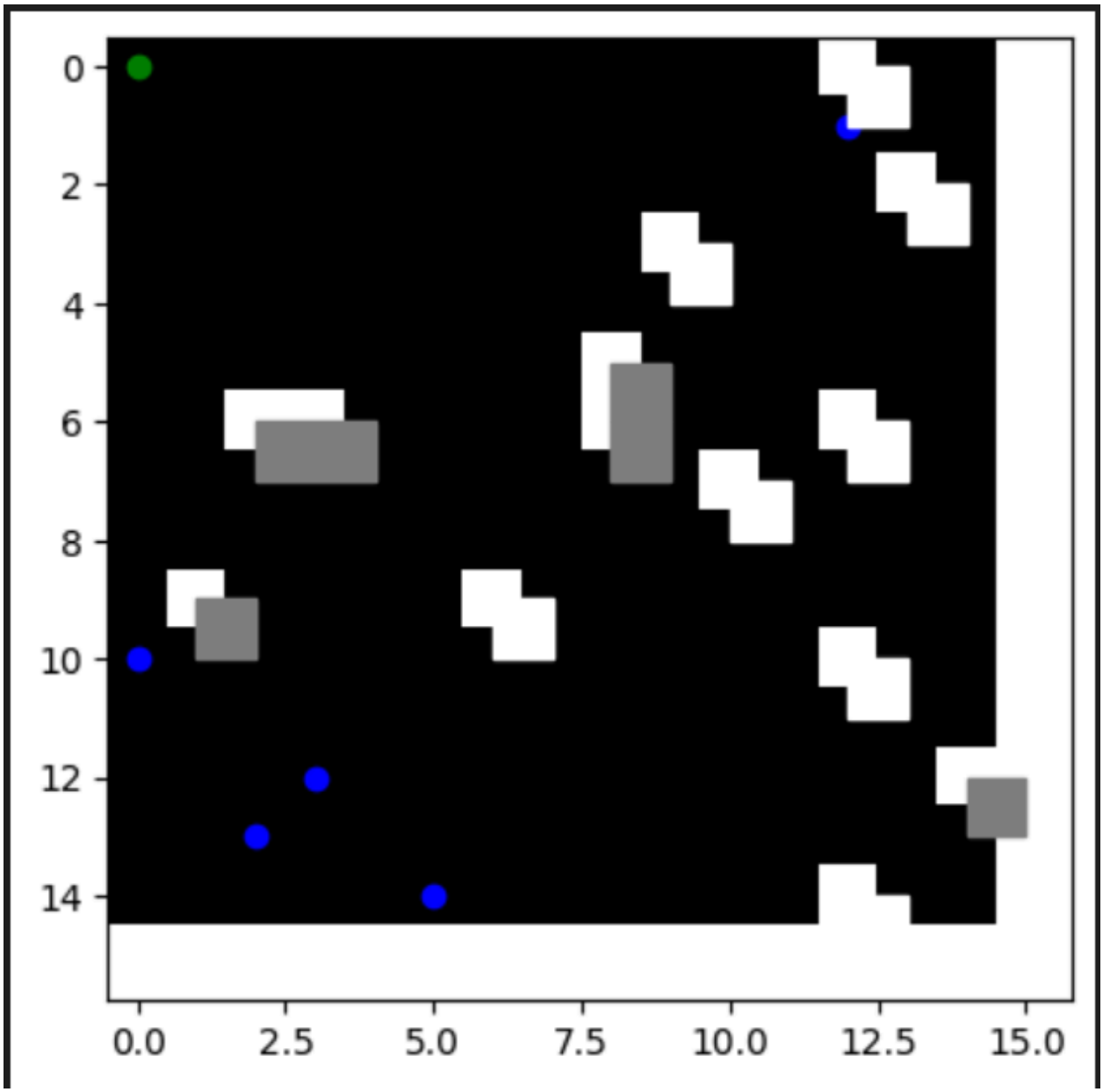
```
    def add_vehicle(self, x, y):
        self.vehicles.append((x, y))

    def update_vehicle_positions(self):
        self.vehicles = [(random.randint(0, self.width - 1), random.randint(0,
self.height - 1)) for _ in range(5)]

class Robot:
    def __init__(self, environment):
        self.environment = environment
        self.current_position = (0, 0)
        self.unvisited_delivery_points = self.environment.delivery_points.copy()

    def plan_path_to_closest_delivery_point(self):
        if self.unvisited_delivery_points:
            # Find the closest unvisited delivery point
            closest_point = min(self.unvisited_delivery_points, key=lambda x:
self.heuristic(self.current_position, x))
            self.goal_position = closest_point
            path = self.a_star_search(self.current_position, self.goal_position)
            return path
        else:
            return None

    def a_star_search(self, start, goal):
        open_list = [(0, start)]
        came_from = {}
        g_score = {start: 0}
        f_score = {start: self.heuristic(start, goal)}

        while open_list:
            current = min(open_list)[1]
            if current == goal:
                path = []
                while current in came_from:
                    path.append(current)
                    current = came_from[current]
                path.append(start)
                path.reverse()
                return path
            open_list.remove((min(open_list)[0], current))
            for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                neighbor = (current[0] + dx, current[1] + dy)
                if 0 <= neighbor[0] < self.environment.height and 0 <=
neighbor[1] < self.environment.width:
                    if self.environment.obstacles[neighbor[1], neighbor[0]]:
                        continue
```

```
                    if neighbor not in g_score or g_score[neighbor] >
g_score[current] + 1:
                        came_from[neighbor] = current
                        g_score[neighbor] = g_score[current] + 1
                        f_score[neighbor] = g_score[neighbor] +
self.heuristic(neighbor, goal)
                        open_list.append((f_score[neighbor], neighbor))
        return None

    def heuristic(self, a, b):
        return np.sqrt((a[0] - b[0])**2 + (a[1] - b[1])**2)

class DynamicEnvironment:
    def __init__(self, environment):
        self.environment = environment
        self.vehicles = []

    def update(self):
        self.environment.vehicles = [(random.randint(0, self.environment.width -
1), random.randint(0, self.environment.height - 1)) for _ in range(5)]

class PathExecution:
    def __init__(self, robot, environment):
        self.robot = robot
        self.environment = environment

    def execute_path(self, path):
        for position in path:
            # Check if the next position is an obstacle
            if self.environment.obstacles[position[1], position[0]]:
                # Avoid collision by moving in the opposite direction
                alternative_path = self.find_alternative_path(position)
                if alternative_path:
                    path = path[:path.index(position)] + alternative_path +
path[path.index(position)+1:]
                else:
                    # If no alternative path is found, stop the robot
                    break
            self.robot.current_position = position

            # Clear the plot
            plt.clf()

            # Plot the environment
            plt.imshow(self.environment.obstacles, cmap='gray')
            plt.scatter([x for x, _ in self.environment.delivery_points], [y for
_, y in self.environment.delivery_points], c='blue')
```

```
            plt.scatter([x for x, _ in self.environment.vehicles], [y for _, y in
self.environment.vehicles], c='red')

            # Plot the current position of the robot
            plt.scatter(self.robot.current_position[0],
self.robot.current_position[1], c='green')

            # Plot the buildings and houses
            for building in self.environment.buildings:
                plt.gca().add_patch(plt.Rectangle((building[0], building[1]), 1,
1, color='white'))
            for house in self.environment.houses:
                plt.gca().add_patch(plt.Rectangle((house[0], house[1]), 1, 1,
color='gray'))

            # Pause to show the plot
            plt.pause(0.1)

    def find_alternative_path(self, position):
        opposite_direction = [(position[0] - 1, position[1]), (position[0] + 1,
position[1]), (position[0], position[1] - 1), (position[0], position[1] + 1)]
        for direction in opposite_direction:
            if 0 <= direction[0] < self.environment.width and 0 <= direction[1] <
self.environment.height:
                if not self.environment.obstacles[direction[1], direction[0]] and
not any(vehicle == direction for vehicle in self.environment.vehicles):
                    return [direction]
        return None

class Visualization:
    def __init__(self, environment, robot):
        self.environment = environment
        self.robot = robot
        self.fig, self.ax = plt.subplots()
        self.plot_environment()

    def plot_environment(self):
        self.ax.clear()
        self.ax.imshow(self.environment.obstacles, cmap='gray')

        for building in self.environment.buildings:
            self.ax.add_patch(plt.Rectangle((building[0], building[1]), 1, 1,
color='white'))

        for house in self.environment.houses:
            self.ax.add_patch(plt.Rectangle((house[0], house[1]), 1, 1,
color='gray'))
```

```
        self.ax.scatter(*zip(*self.environment.delivery_points), c='blue',
marker='o', label='Delivery Points')

        if self.environment.vehicles:
            vehicle_x, vehicle_y = zip(*self.environment.vehicles)
            self.ax.scatter(vehicle_x, vehicle_y, c='red', marker='s',
label='Vehicles')

        if hasattr(self.environment, 'start_position'):
            self.ax.scatter(*self.environment.start_position, c='green',
marker='*', label='Start Position')

        self.ax.scatter(self.robot.current_position[0],
self.robot.current_position[1], c='yellow', marker='v', label='Robot')

        self.ax.set_xlim([0, self.environment.width])
        self.ax.set_ylim([0, self.environment.height])
        self.ax.set_aspect('equal', 'box')
        self.ax.legend()
        plt.draw()
        plt.pause(0.1)

    def update(self):
        self.plot_environment()

class PerformanceEvaluation:
    def __init__(self, robot, environment):
        self.robot = robot
        self.environment = environment

    def evaluate(self):
        path = self.robot.plan_path_to_closest_delivery_point()
        if path:
            path_execution = PathExecution(self.robot, self.environment)
            path_execution.execute_path(path)
            print("Path Optimality:", len(path))
            print("Execution Time:", len(path))
            print("Adaptability to Dynamic Changes:",
len(self.environment.vehicles))
            self.robot.unvisited_delivery_points.remove(self.robot.goal_position)
        else:
            print("All delivery points have been visited.")

if __name__ == "__main__":
    environment = Environment(15, 15)
    delivery_locations = [(random.randint(0, environment.width - 1),
random.randint(0, environment.height - 1)) for _ in range(5)]
    for x, y in delivery_locations:
```

```
        environment.add_delivery_point(x, y)
    robot = Robot(environment)
    dynamic_environment = DynamicEnvironment(environment)
    path_execution = PathExecution(robot, environment)
    visualization = Visualization(environment, robot)
    performance_evaluation = PerformanceEvaluation(robot, environment)

    while robot.unvisited_delivery_points:
        path = robot.plan_path_to_closest_delivery_point()
        if path:
            path_execution.execute_path(path)
            performance_evaluation.evaluate()
            robot.current_position = robot.goal_position
            visualization.update()  # Update the visualization
        else:
            break
```



Task_2:

```
import tkinter as tk
from tkinter import ttk, messagebox
import time
import random

class SudokuGUI:
```

```
    def __init__(self, master):
        self.master = master
        master.title("Sudoku")
        self.master.resizable(False, False)  # Prevent resizing of the window
        self.board = [[tk.StringVar() for _ in range(9)] for _ in range(9)]
        self.cells = [[None for _ in range(9)] for _ in range(9)]
        self.start_time = None
        self.difficulty_levels = {
            "Easy": 0.3,
            "Medium": 0.5,
            "Hard": 0.7
        }

        # Styling
        self.style = ttk.Style()
        self.style.theme_use('clam')
        self.style.configure("TButton", font=('Arial', 12), padding=8,
background='blue', foreground='white')
        self.style.configure("TEntry", font=('Arial', 16), padding=8,
fieldbackground='red')
        self.style.configure("TLabel", font=('Arial', 16), foreground='black')
        self.style.configure("Level.TLabel", font=('Arial', 14),
foreground='orange', background='yellow')

        # Level Selection
        self.level_var = tk.StringVar(value="Easy")
        level_frame = ttk.Frame(master)
        level_frame.grid(row=0, column=0, columnspan=3, pady=(10, 5))
        ttk.Label(level_frame, text="Level:", style="Level.TLabel").grid(row=0,
column=0, padx=5, sticky="e")
        self.level_menu = ttk.OptionMenu(level_frame, self.level_var, "Easy",
*self.difficulty_levels.keys(), command=self.change_level)
        self.level_menu.grid(row=0, column=1, padx=5, sticky="ew")

        # Sudoku Grid Frame
        self.grid_frame = ttk.Frame(master)
        self.grid_frame.grid(row=1, column=0, columnspan=3, padx=10, pady=5)

        # Create grid of cells
        for i in range(9):
            for j in range(9):
                bg_color = '#f0f0f0' if (i // 3 + j // 3) % 2 == 0 else '#e0e0e0'
                cell = ttk.Entry(self.grid_frame, width=2, font=('Arial', 18),
justify='center', textvariable=self.board[i][j], background=bg_color)
                cell.grid(row=i, column=j, stick="nsew", padx=1, pady=1)
                self.cells[i][j] = cell

                # Add borders to create 3x3 grid layout
```

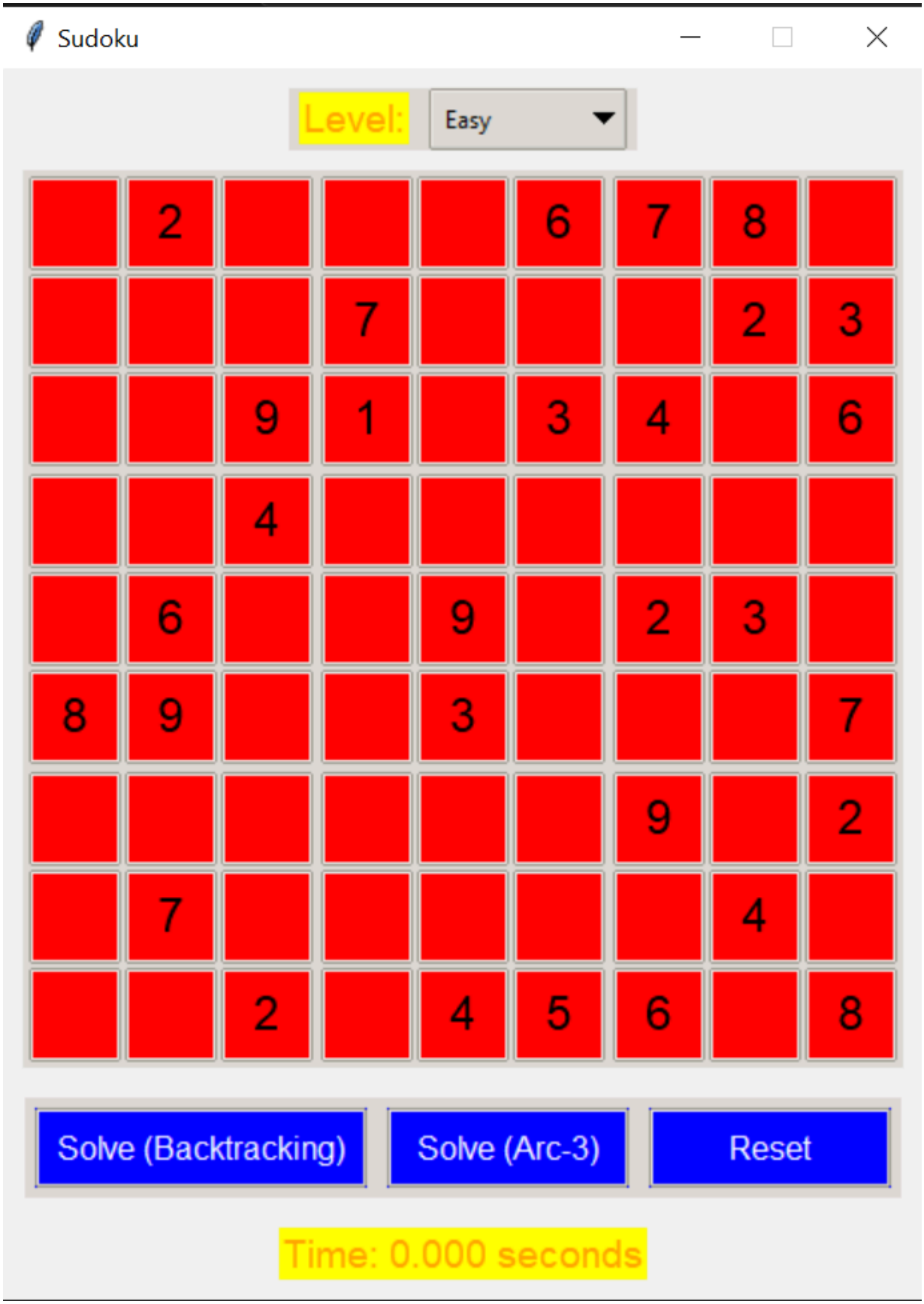
```
                if i % 3 == 0:
                    cell.grid(pady=(3, 1))  # Add top border
                if j % 3 == 0:
                    cell.grid(padx=(3, 1))  # Add left border
                if i == 8:
                    cell.grid(pady=(1, 3))  # Add bottom border
                if j == 8:
                    cell.grid(padx=(1, 3))  # Add right border

        # Solve and Reset Buttons
        solve_frame = ttk.Frame(master)
        solve_frame.grid(row=2, column=0, columnspan=3, padx=10, pady=10)
        ttk.Button(solve_frame, text="Solve (Backtracking)",
command=self.solve_backtracking).grid(row=0, column=0, padx=5, pady=5)
        ttk.Button(solve_frame, text="Solve (Arc-3)",
command=self.solve_arc3).grid(row=0, column=1, padx=5, pady=5)
        ttk.Button(solve_frame, text="Reset", command=self.reset).grid(row=0,
column=2, padx=5, pady=5)

        # Timer Label
        self.timer_label = ttk.Label(master, text="Time: 0.000 seconds",
style="Level.TLabel")
        self.timer_label.grid(row=3, column=0, columnspan=3, pady=(5, 10))

        # Initialize a random game
        self.generate_random_state()

    def change_level(self, level):
        self.generate_random_state()

    def generate_random_state(self):
        difficulty = self.difficulty_levels[self.level_var.get()]
        self.board = [[tk.StringVar() for _ in range(9)] for _ in range(9)]

        # Generate a solved puzzle
        solved = [[(i * 3 + i // 3 + j) % 9 + 1 for j in range(9)] for i in
range(9)]
        self.board = [[tk.StringVar(value=str(solved[i][j]) if random.random() <
difficulty else "") for j in range(9)] for i in range(9)]

        self.refresh_board()

    def refresh_board(self):
        for i in range(9):
            for j in range(9):
                self.cells[i][j].delete(0, tk.END)
                self.cells[i][j].insert(0, self.board[i][j].get())
```

```
    def reset(self):
        self.generate_random_state()

    def solve_backtracking(self):
        self.start_time = time.time()
        if self.solve_sudoku_backtracking():
            elapsed_time = time.time() - self.start_time
            self.timer_label.config(text=f"Time: {elapsed_time:.3f} seconds")
            messagebox.showinfo("Sudoku Solver (Backtracking)", "Puzzle solved!",
icon='info')
            self.refresh_board()  # Refresh the board after solving
        else:
            elapsed_time = time.time() - self.start_time
            self.timer_label.config(text=f"Time: {elapsed_time:.3f} seconds")
            messagebox.showinfo("Sudoku Solver (Backtracking)", "No solution
exists. Showing best attempt.", icon='warning')
            self.highlight_conflicts()

    def solve_arc3(self):
        self.start_time = time.time()
        if self.solve_sudoku_arc3():
            elapsed_time = time.time() - self.start_time
            self.timer_label.config(text=f"Time: {elapsed_time:.3f} seconds")
            messagebox.showinfo("Sudoku Solver (Arc-3)", "Puzzle solved!",
icon='info')
            self.refresh_board()  # Refresh the board after solving
        else:
            elapsed_time = time.time() - self.start_time
            self.timer_label.config(text=f"Time: {elapsed_time:.3f} seconds")
            messagebox.showinfo("Sudoku Solver (Arc-3)", "No solution exists.
Showing best attempt.", icon='warning')
            self.highlight_conflicts()

    def highlight_conflicts(self):
        for i in range(9):
            for j in range(9):
                if not self.is_valid(i, j, self.board[i][j].get()):
                    self.cells[i][j].config(foreground='red')

    def solve_sudoku_backtracking(self):
        # Backtracking algorithm
        empty = self.find_empty_location()
        if not empty:
            return True
        row, col = empty

        for num in range(1, 10):
            if self.is_valid(row, col, str(num)):
```

```
                self.board[row][col].set(str(num))
                if self.solve_sudoku_backtracking():
                    return True
                self.board[row][col].set("")
        return False

    def solve_sudoku_arc3(self):
        # Arc-3 algorithm
        # Initialize the domain for each cell
        domain = {}
        for i in range(9):
            for j in range(9):
                if self.board[i][j].get() == "":
                    domain[(i, j)] = set(str(num) for num in range(1, 10))

        # Define the constraints for each cell
        constraints = {}
        for i in range(9):
            for j in range(9):
                if self.board[i][j].get() == "":
                    constraints[(i, j)] = set()
                    # Add constraints for the row and column
                    for k in range(9):
                        if k != j:
                            constraints[(i, j)].add((i, k))
                        if k != i:
                            constraints[(i, j)].add((k, j))
                    # Add constraints for the 3x3 subgrid
                    start_row, start_col = 3 * (i // 3), 3 * (j // 3)
                    for k in range(3):
                        for l in range(3):
                            if start_row + k != i or start_col + l != j:
                                constraints[(i, j)].add((start_row + k, start_col
+ l))

        # Perform arc consistency
        queue = list(domain.keys())
        while queue:
            cell_i, cell_j = queue.pop(0)
            for constraint_i, constraint_j in constraints[(cell_i, cell_j)]:
                if (constraint_i, constraint_j) != (cell_i, cell_j):
                    if len(domain.get((constraint_i, constraint_j), set())) == 1:
                        value = next(iter(domain.get((constraint_i,
constraint_j), set())))
                        if value in domain.get((cell_i, cell_j), set()):
                            domain[(cell_i, cell_j)].remove(value)
                            if len(domain[(cell_i, cell_j)]) == 0:
                                return False
```

```
                            queue.append((cell_i, cell_j))

        # Backtrack with the arc-3 algorithm
        return self.arc3_backtrack(domain)

    def arc3_backtrack(self, domain):
        # Backtracking with the arc-3 algorithm
        empty = self.find_empty_location()
        if not empty:
            return True
        row, col = empty

        for value in domain[(row, col)]:
            if self.is_valid(row, col, value):
                self.board[row][col].set(value)
                if self.solve_sudoku_arc3():
                    return True
                self.board[row][col].set("")
        return False

    def find_empty_location(self):
        for i in range(9):
            for j in range(9):
                if self.board[i][j].get() == "":
                    return i, j
        return None

    def is_valid(self, row, col, num):
        for i in range(9):
            if self.board[i][col].get() == num or self.board[row][i].get() ==
num:
                return False

        start_row, start_col = 3 * (row // 3), 3 * (col // 3)
        for i in range(3):
            for j in range(3):
                if self.board[start_row + i][start_col + j].get() == num:
                    return False

        return True

def main():
    root = tk.Tk()
    app = SudokuGUI(root)
    root.mainloop()

if __name__ == "__main__":
    main()
```

[redacted]


Sudoku
Level:
Easy
2 6 7 8
7 2 3
9 1 3 4 6
4
6 9 2 3
8 9 3 7
9 2
7 4
2 4 5 6 8
Solve (Backtracking)
Solve (Arc-3)
Reset
Time: 0.000 seconds

Task_3:

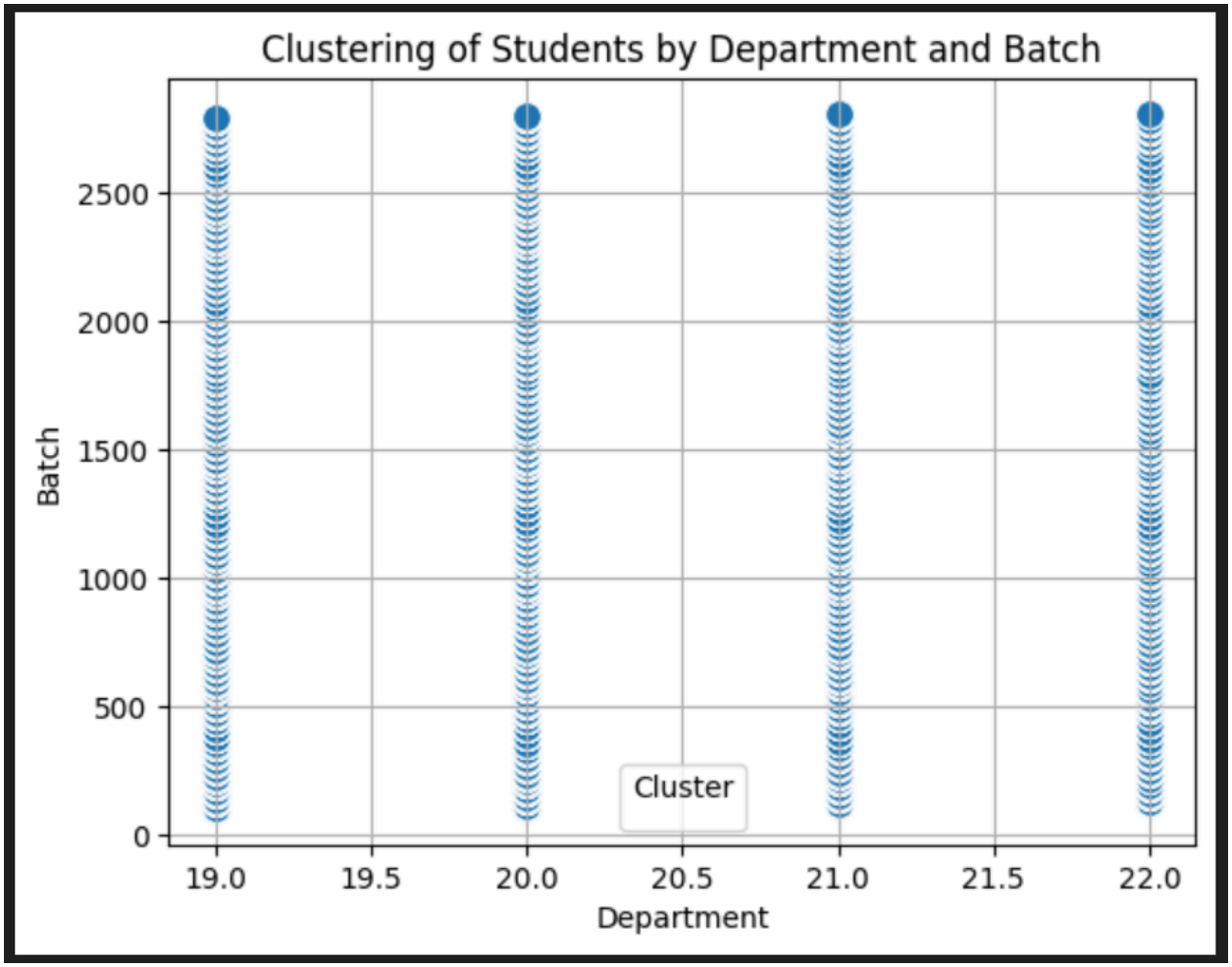
```
import pandas as pd
from sklearn.cluster import KMeans

# Data Collection
teachers_df = pd.read_csv("Teachers.csv")
students_df = pd.read_csv("Students.csv")
rooms_df = pd.read_csv("Rooms.csv")

# Filter students from batches 19, 20, 21, and 22
filtered_students_df = students_df[students_df['Batch'].isin([19, 20, 21, 22])]

# Count number of students in each domain and batch
student_counts = filtered_students_df.groupby(['Dept',
'Batch']).size().reset_index(name='Count')

# K-Means Clustering
X = student_counts[['Dept', 'Batch', 'Count']]
kmeans = KMeans(n_clusters=5)  # You need to determine the optimal number of
clusters
kmeans.fit(X)
student_counts['Cluster'] = kmeans.labels_

# Seating Plan Generation (Sample Implementation)
def generate_seating_plan(student_counts, rooms_df):
    # Your implementation here
    # This is just a placeholder implementation
    seating_plan = {}
    for index, row in rooms_df.iterrows():
        room_id = row['RoomID']
        room_capacity = row['Capacity']
        students_in_room = student_counts[student_counts['Cluster'] == index]
        seating_plan[room_id] = students_in_room
    return seating_plan

seating_plan = generate_seating_plan(student_counts, rooms_df)

# Faculty Allocation (Sample Implementation)
def allocate_faculty(student_counts, teachers_df):
    # Your implementation here
    # This is just a placeholder implementation
    faculty_allocation = {}
    for index, row in student_counts.iterrows():
        cluster = row['Cluster']
        faculty_for_cluster = teachers_df[teachers_df['Dept'] == row['Dept']]
        faculty_allocation[cluster] = faculty_for_cluster
```

```
    return faculty_allocation

faculty_allocation = allocate_faculty(student_counts, teachers_df)

# Reporting (Sample Implementation)
def generate_report(seating_plan, faculty_allocation):
    # Your implementation here
    # This is just a placeholder implementation
    report = "Seating Plan:\n"
    for room_id, students in seating_plan.items():
        report += f"Room {room_id}: {students}\n"
    report += "\nFaculty Allocation:\n"
    for cluster, faculty in faculty_allocation.items():
        report += f"Cluster {cluster}: {faculty}\n"
    return report

report = generate_report(seating_plan, faculty_allocation)

# Display report
print(report)
```

```
...   Seating Plan:
      Room 1:      Dept  Batch  Count  Cluster
      2       0     21    176         0
      3       0     22    176         0
      6       1     21    176         0
      10      2     21    176         0
      Room 2:      Dept  Batch  Count  Cluster
      11      2     22    132         1
      15      3     22    132         1
      Room 3:     Dept  Batch  Count  Cluster
      7      1     22    154         2
      Room 4:     Dept  Batch  Count  Cluster
      0      0     19    176         3
      1      0     20    176         3
      4      1     19    176         3
      5      1     20    176         3
      Room 5:      Dept  Batch  Count  Cluster
      8       2     19    176         4
      9       2     20    176         4
      12      3     19    176         4
      13      3     20    176         4
      14      3     21    176         4
      Room 6: Empty DataFrame
      Columns: [Dept, Batch, Count, Cluster]
      Index: []
      ...
      Cluster 1: Empty DataFrame
      Columns: [Dept, Name]
```

```
pip install reportlab
import pandas as pd
from sklearn.cluster import KMeans
```

```
import matplotlib.pyplot as plt
import seaborn as sns
from reportlab.lib.pagesizes import letter
from reportlab.platypus import SimpleDocTemplate, Paragraph, Table, TableStyle
from reportlab.lib import colors
from reportlab.lib.styles import getSampleStyleSheet

# Data Collection
teachers_df = pd.read_csv("Teachers.csv")
students_df = pd.read_csv("Students.csv")
rooms_df = pd.read_csv("Rooms.csv")



# Filter students from batches 19, 20, 21, and 22
filtered_students_df = students_df[students_df['Batch'].isin([19, 20, 21, 22])]

# Count number of students in each domain and batch
student_counts = filtered_students_df.groupby(['Dept',
'Batch']).size().reset_index(name='Count')

# K-Means Clustering
X = students_df[['Batch','Dept','ID']]
kmeans = KMeans(n_clusters=5)  # You need to determine the optimal number of
clusters
kmeans.fit(X)
#student_counts['Cluster'] = kmeans.labels_

# Visualization
plt.figure()
sns.scatterplot(data=students_df, x='Batch', y='ID', palette='viridis', s=100)
plt.title('Clustering of Students by Department and Batch')
plt.xlabel('Department')
plt.ylabel('Batch')
plt.legend(title='Cluster')
plt.grid(True)
plt.savefig('Student_Cluster_Visualization.png')
plt.show()
```




```python
import pandas as pd
from sklearn.cluster import KMeans
import random

teacher_id_to_name = {
    1: "John Smith",
    2: "Alice Johnson",
    3: "Michael Brown",
    4: "Emily Davis",
    5: "James Wilson",
    6: "Sarah Clark",
    7: "David Martinez",
    8: "Jessica Thompson",
    9: "Robert Garcia",
    10: "Emma White",
    11: "Daniel L1",
    12: "Olivia Hall",
    13: "William Rodriguez",
    14: "Sophia Lopez",
    15: "Matthew Perez",
```

```
    16: "Isabella Gr1n",
    17: "Joseph Carter",
    18: "Abig2l Young",
    19: "Andrew Hill",
    20: "Mia King",
    21: "Alexander Scott",
    22: "Charlotte Evans",
    23: "Daniel Turner",
    24: "Emily Moore",
    25: "David Harris",
    26: "Samantha Allen",
    27: "Christopher Baker",
    28: "Lily Wright",
    29: "Matthew Nelson"
}
 # Your teacher ID to name mapping
department_id_to_name = {0: "CS", 1: "EE", 2: "AI", 3: "BBA"}  # Your department
ID to name mapping

# Data Collection
teachers_df = pd.read_csv("Teachers.csv")
students_df = pd.read_csv("Students.csv")
rooms_df = pd.read_csv("Rooms.csv")

# Function to generate dummy date sheet
def generate_date_sheet(students_df):
    date_sheet = []
    for _, student in students_df.iterrows():
        date = f"2024-05-{random.randint(1, 30)}"  # Random date in May 2024
        subject = f"Subject_{random.randint(1, 5)}"  # Random subject
        dept = student['Dept']  # Corrected department column name
        batch = student['Batch']
        time_slot = random.choice(["Morning", "Afternoon"])  # Random time slot
        date_sheet.append([date, subject, dept, batch, time_slot])
    return pd.DataFrame(date_sheet, columns=['Date', 'Subject', 'Dept', 'Batch',
'Time Slot'])

# Generate Dummy Date Sheet
dummy_date_sheet = generate_date_sheet(students_df)
print("\nDummy Date Sheet:")
print(dummy_date_sheet.head())  # Print first few rows of the date sheet

# K-Means Clustering
students_df['Batch'] = pd.to_numeric(students_df['Batch'], errors='coerce')  #
Convert Batch to numeric
X = students_df[['Batch', 'Dept']]
kmeans = KMeans(n_clusters=len(rooms_df))
kmeans.fit(X)
```

```
students_df['Cluster'] = kmeans.labels_

# Seating Plan Generation
seating_plan = {}
for index, room in rooms_df.iterrows():
    room_id = room['RoomID']
    capacity = room['Capacity']
    students_in_room = students_df[students_df['Cluster'] == index]
    seating_plan[room_id] = students_in_room.head(capacity)  # Assign students up
to room capacity

# Faculty Allocation
faculty_allocation = {}
for cluster, students in students_df.groupby('Cluster'):
    dept = students['Dept'].iloc[0]  # Assuming all students in a cluster belong
to the same department
    faculty_allocation[cluster] = teachers_df[teachers_df['Dept'] == dept]

# # Print Seating Plan

print("\nSeating Plan:")
for room_id, students in seating_plan.items():
    print(f"Room {room_id}:")
    if not students.empty:  # Check if there are any students in the room
        invigilator_id =
faculty_allocation[students['Cluster'].iloc[0]]['TeacherID'].iloc[0]
        faculty_name = teacher_id_to_name[invigilator_id]
        for i, (_, student) in enumerate(students.iterrows(), start=1):
            print(f"Seat Number: {i}, Student ID: {student['ID']}, Batch:
{student['Batch']}, Dept: {department_id_to_name[student['Dept']]}, "
                  f"Time Slot: {dummy_date_sheet.loc[dummy_date_sheet['Dept'] ==
student['Dept'], 'Time Slot'].iloc[0]}, "
                  f"Subject: {dummy_date_sheet.loc[dummy_date_sheet['Dept'] ==
student['Dept'], 'Subject'].iloc[0]}, "
                  f"Invigilator: {faculty_name}")
    else:
        print("No students assigned to this room.")
```

```
Dummy Date Sheet:
         Date    Subject  Dept  Batch  Time Slot
0  2024-05-23  Subject_1     0     19  Afternoon
1  2024-05-23  Subject_3     0     19    Morning
2   2024-05-2  Subject_5     1     19    Morning
3  2024-05-15  Subject_1     1     19    Morning
4  2024-05-14  Subject_2     2     19    Morning

Seating Plan:
Room 1:
Seat Number: 1, Student ID: 121, Batch: 21, Dept: AI, Time Slot: Morning, Subject: Subject_2, Invigilator: James Wilson
Seat Number: 2, Student ID: 122, Batch: 21, Dept: AI, Time Slot: Morning, Subject: Subject_2, Invigilator: James Wilson
Seat Number: 3, Student ID: 153, Batch: 21, Dept: AI, Time Slot: Morning, Subject: Subject_2, Invigilator: James Wilson
Seat Number: 4, Student ID: 154, Batch: 21, Dept: AI, Time Slot: Morning, Subject: Subject_2, Invigilator: James Wilson
Seat Number: 5, Student ID: 185, Batch: 21, Dept: AI, Time Slot: Morning, Subject: Subject_2, Invigilator: James Wilson
Seat Number: 6, Student ID: 186, Batch: 21, Dept: AI, Time Slot: Morning, Subject: Subject_2, Invigilator: James Wilson
Seat Number: 7, Student ID: 217, Batch: 21, Dept: AI, Time Slot: Morning, Subject: Subject_2, Invigilator: James Wilson
Seat Number: 8, Student ID: 218, Batch: 21, Dept: AI, Time Slot: Morning, Subject: Subject_2, Invigilator: James Wilson
Seat Number: 9, Student ID: 244, Batch: 21, Dept: AI, Time Slot: Morning, Subject: Subject_2, Invigilator: James Wilson
Seat Number: 10, Student ID: 245, Batch: 21, Dept: AI, Time Slot: Morning, Subject: Subject_2, Invigilator: James Wilson
Seat Number: 11, Student ID: 276, Batch: 21, Dept: AI, Time Slot: Morning, Subject: Subject_2, Invigilator: James Wilson
Seat Number: 12, Student ID: 277, Batch: 21, Dept: AI, Time Slot: Morning, Subject: Subject_2, Invigilator: James Wilson
Seat Number: 13, Student ID: 308, Batch: 21, Dept: AI, Time Slot: Morning, Subject: Subject_2, Invigilator: James Wilson
Seat Number: 14, Student ID: 309, Batch: 21, Dept: AI, Time Slot: Morning, Subject: Subject_2, Invigilator: James Wilson
...
Seat Number: 15, Student ID: 336, Batch: 21, Dept: CS, Time Slot: Afternoon, Subject: Subject_1, Invigilator: John Smith
Seat Number: 16, Student ID: 337, Batch: 21, Dept: CS, Time Slot: Afternoon, Subject: Subject_1, Invigilator: John Smith
Seat Number: 17, Student ID: 363, Batch: 21, Dept: CS, Time Slot: Afternoon, Subject: Subject_1, Invigilator: John Smith
```

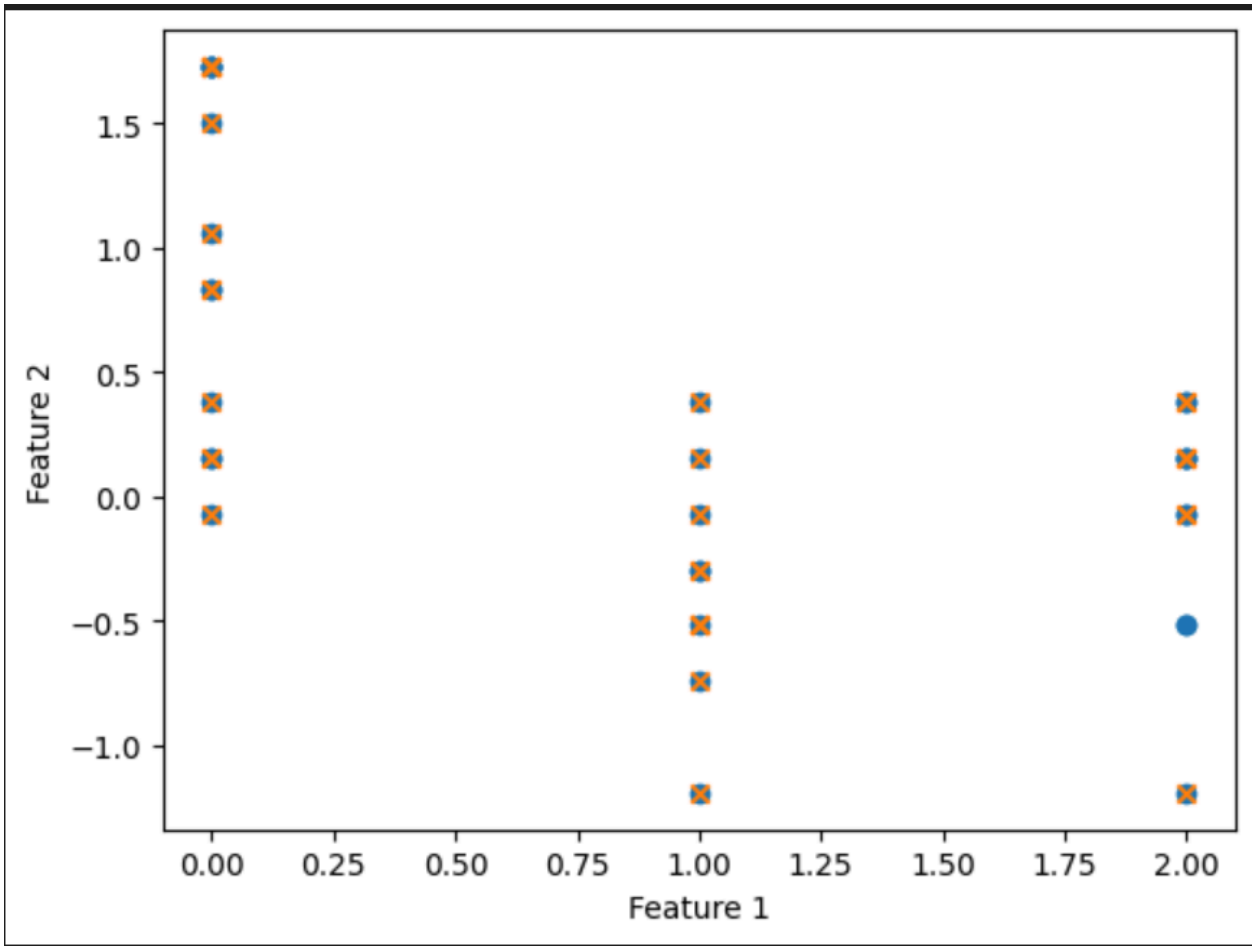
Task_4:

```python
from ucimlrepo import fetch_ucirepo

import pandas as pd

# iris = fetch_ucirepo(id=53)


# X = iris.data.features
# y = iris.data.targets

# df = pd.DataFrame(X)
# df['class'] = iris.data.targets

# df.to_csv('Iris.csv', index=False)
# one_hot_encoded_data = pd.get_dummies(df, columns = ['class'])
# one_hot_encoded_data.to_csv('IrisEncoded.csv', index=False)
# print(one_hot_encoded_data)
data = pd.read_csv('Iris.csv')

data = pd.read_csv('Iris.csv')
X = data.iloc[:, :4]
y =   data.iloc[:, 4]
import numpy as np
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```python
import numpy as np
S1 = []
class singleLayerNN:

    def __init__(self, num_inputs, learning_rate=0.1):
        self.weights = np.random.rand(num_inputs + 1)
        self.learning_rate = learning_rate


    def linear(self, inputs):
        Z = inputs @ self.weights[1:].T  + self.weights[0]

        return Z


    def activation(self, z):
        if z <= 0:
            return 0
        elif z>0 and z<= 1:
            return 1
        else:
            return 2


    def predict(self, inputsTest):
        Z = self.linear(inputsTest)
        try:
            pred = []
            for z in Z:
                pred.append(self.activation(z))
        except:
            return self.activation(Z)
        return pred



    def train(self, inputsTrain, target):
        Z = self.linear(inputsTrain)
        y_pred = self.predict(inputsTrain)
        S1.append(y_pred)
        self.weights[1:] += self.learning_rate * (target-y_pred) * inputsTrain
        self.weights[0] += self.learning_rate *(target-y_pred)
        print('loss: ',target-Z)


    def fit(self, X, y, iter):
```

```
        for _ in range(iter):
            for inputs, target in zip(X, y):
                self.train(inputs, target)

X_train, X_test, y_train, y_test =
train_test_split(X,y,test_size=0.2,random_state=23,shuffle=True)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

np.random.seed(23)
num_inputs=X_train.shape[1]
print('num_inputs',num_inputs)
perceptron = singleLayerNN(num_inputs)


perceptron.fit(X_train, y_train,1000)


pred = perceptron.predict(X_test)

accuracy = np.mean(pred == y_test)
print("Accuracy:", accuracy)


count = 0
for k,i,j in zip(X_test,pred,y_test):
    if (i != j):
        print(k,'  ',i,'  ',j)
        count+=1
print(count)
print(len(y_test))
plt.scatter(y_test, X_test[:, 1])
plt.scatter(pred, X_test[:, 1],marker='x')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

```
...   num_inputs 4
      loss:  1.126761050644931
      loss:  1.7038604621355473
      loss:  -0.8377155505199898
      loss:  -1.4235034273789786
      loss:  1.0403190963875355
      loss:  -0.1057453552426193
      loss:  -0.4733013007371407
      loss:  2.4258509648224034
      loss:  0.6227091831700307
      loss:  -0.2508291875975077
      loss:  -0.5977013371414648
      loss:  1.466506255635681
      loss:  1.3088020242911873
      loss:  1.7260348986382472
      loss:  -0.06834871678769772
      loss:  0.8184072652217451
      loss:  2.2166049121619142
      loss:  -1.0153870373309681
      loss:  1.3125830239995422
      loss:  0.3028550193261319
      loss:  -0.6399710058989232
      loss:  0.03099679185108717
      loss:  1.0040891586737226
      loss:  0.5567258471729777
      ...
      [ 0.56606881 -0.51675608  0.74211553  0.37239251]    1    2
      [ 0.44520002 -0.51675608  0.57107738  0.77020054]    1    2
```




```
import numpy as np
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

class MLP:
    def __init__(self, num_inputs, num_hidden1, num_hidden2, num_outputs,
learning_rate=0.1):
        self.weights_input_hidden1 = np.random.randn(num_inputs, num_hidden1)
        self.bias_hidden1 = np.zeros((1, num_hidden1))
        self.weights_hidden1_hidden2 = np.random.randn(num_hidden1, num_hidden2)
        self.bias_hidden2 = np.zeros((1, num_hidden2))
        self.weights_hidden2_output = np.random.randn(num_hidden2, num_outputs)
        self.bias_output = np.zeros((1, num_outputs))
        self.learning_rate = learning_rate

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))
```

```
    def forward_pass(self, X):
        self.hidden1_input = np.dot(X, self.weights_input_hidden1) +
self.bias_hidden1
        self.hidden1_output = self.sigmoid(self.hidden1_input)
        self.hidden2_input = np.dot(self.hidden1_output,
self.weights_hidden1_hidden2) + self.bias_hidden2
        self.hidden2_output = self.sigmoid(self.hidden2_input)
        self.output_input = np.dot(self.hidden2_output,
self.weights_hidden2_output) + self.bias_output
        self.output_output = self.sigmoid(self.output_input)

    def backward_pass(self, X, y):
        num_examples = X.shape[0]
        d_output = self.output_output - y
        d_hidden2_output = np.dot(d_output, self.weights_hidden2_output.T) *
(self.hidden2_output * (1 - self.hidden2_output))
        d_hidden1_output = np.dot(d_hidden2_output,
self.weights_hidden1_hidden2.T) * (self.hidden1_output * (1 -
self.hidden1_output))

        self.weights_hidden2_output -= self.learning_rate *
np.dot(self.hidden2_output.T, d_output) / num_examples
        self.bias_output -= self.learning_rate * np.sum(d_output, axis=0,
keepdims=True) / num_examples
        self.weights_hidden1_hidden2 -= self.learning_rate *
np.dot(self.hidden1_output.T, d_hidden2_output) / num_examples
        self.bias_hidden2 -= self.learning_rate * np.sum(d_hidden2_output,
axis=0, keepdims=True) / num_examples
        self.weights_input_hidden1 -= self.learning_rate * np.dot(X.T,
d_hidden1_output) / num_examples
        self.bias_hidden1 -= self.learning_rate * np.sum(d_hidden1_output,
axis=0, keepdims=True) / num_examples

    def train(self, X, y, epochs):
        for i in range(epochs):
            self.forward_pass(X)
            self.backward_pass(X, y)

    def predict(self, X):
        self.forward_pass(X)
        return np.argmax(self.output_output, axis=1)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=23, shuffle=True)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
np.random.seed(23)
num_inputs = X_train.shape[1]
num_hidden1 = 8  # Number of neurons in the first hidden layer
num_hidden2 = 4  # Number of neurons in the second hidden layer
num_outputs = 3  # Number of output classes
learning_rate = 0.1
epochs = 1000

mlp = MLP(num_inputs, num_hidden1, num_hidden2, num_outputs, learning_rate)

mlp.train(X_train, np.eye(num_outputs)[y_train], epochs)

pred = mlp.predict(X_test)

accuracy = np.mean(pred == y_test)
print("Accuracy:", accuracy)

count = np.sum(pred != y_test)
print("Misclassified samples:", count)
print("Total samples:", len(y_test))

plt.scatter(y_test, X_test[:, 1])
plt.scatter(pred, X_test[:, 1], marker='x')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

```
Accuracy: 0.9666666666666667
Misclassified samples: 1
Total samples: 30
```